

Practically Groovy: MVC programming with Groovy templates

Simplify report views with Groovy's template engine framework

Skill Level: Intermediate

[Andrew Glover \(andrew@thirstyhead.com\)](mailto:andrew@thirstyhead.com)

Co-Founder

ThirstyHead.com

[Scott Davis \(scott@thirstyhead.com\)](mailto:scott@thirstyhead.com)

Founder

ThirstyHead.com

15 Feb 2005

Views are an integral part of MVC programming, which is itself a ubiquitous component of enterprise application development. In this installment of Practically Groovy, Andrew Glover shows how Groovy's template engine framework can simplify view programming and make your code more maintainable over time.

I've shown in recent articles in the *Practically Groovy* series that Groovy is a superb tool for constructing reporting applications. I used a checksum reporting application example to show you the basics of [Ant scripting with Groovy](#) and a database statistics report to demonstrate the benefits of [JDBC programming with GroovySql](#). Both of these example reports were generated inside a Groovy script and both, by nature, had a "view" associated with them.

In the case of the checksum reporting example, the view code was somewhat messy. Worse yet, it could prove difficult to maintain: If I ever wanted to change a particular aspect of the view, I would have to modify the code inside the script. So this month, I'll show you how to simplify report views using Groovy's template engine and the checksum reporting application from my previous article.

Template engines are like XSLT and can produce any format for which a template is

defined, including XML, HTML, SQL, and Groovy code. Much like JSPs, template engines make it easier to separate the concerns of a view into a separate entity, such as a JSP or a template file. For example, if a report's intended output was XML, you could create an XML template that contained placeholders for run-time-substituted values. A template engine would then transform the template by reading it and mapping placeholders with run-time values. The output of the process would be an XML-formatted document.

About this series

The key to incorporating any tool into your development practice is knowing when to use it and when to leave it in the box. Scripting languages can be an extremely powerful addition to your tool kit, but only when applied properly to appropriate scenarios. To that end, *Practically Groovy* is a series of articles dedicated to exploring the practical uses of Groovy and teaching you when and how to apply them successfully.

Before we jump into Groovy templates, I'd like to review the notion of `String` types in Groovy. All scripting languages attempt to make string usage exceptionally easy. And, once again, Groovy doesn't let us down. Click the **Code** icon at the top or bottom of this page (or see the [Download](#) section) to download the article source before we get started.

Can't get enough of Strings

Strings in plain Java™ code are, unfortunately, quite limited, although the Java 5.0 application promises some nifty new features. Groovy solves two of the worst Java `String` limitations today, making it easy to write multiline strings and to do run-time substitutions. A few simple examples will bring you up to speed on the Groovy `String` types I'll be working with in this article.

If you wrote a multiline `String` in plain Java code, you would end up with a lot of pesky `+s`, right? Listing 1 demonstrates this:

Listing 1. Multiline strings in Java

```
String example1 = "This is a multiline " +
    "string which is going to " +
    "cover a few lines then " +
    "end with a period.";

String example2 = "To preserve the line-breaks, \n" +
    "you must explicitly include them \n" +
    "with a backslash \"n\".";

String example3 = "You also must \"escape\" internal quotes.";
```

Writing a multiline String in Groovy is much easier. Groovy supports the notion of here-docs, as shown in Listing 2. A *here-doc* is a convenient mechanism for creating formatted Strings, such as HTML and XML. To create a here-doc, simply surround your String with Python-like triple quotes.

Listing 2. Here-docs in Groovy

```
String itext = """This is another multiline String
that takes up a few lines.

Heredocs are different than Java Strings
in a few important ways:
1. They
   preserve
   newlines.

2. You don't have to "escape" internal quotes.
   """
```

Groovy uses GStrings to facilitate run-time substitution. If you're wondering what they are, let me assure you that you've seen them before, and you've probably even used them. Simply put, GStrings allow you to substitute values using the *bash-like* `${}` syntax. The beauty of GStrings is that you don't ever have to know you're using a GString type; you simply code Strings in Groovy as you would in your Java code.

About template engines

Template engines have been around for a long time and can be found in almost every modern language. Normal Java language has Velocity and FreeMarker, to name two; Python has Cheetah and Ruby ERB; and Groovy has its own engine. See [Resources](#) to learn more about template engines.

Listing 3. GStrings in Groovy

```
def lang = "Groovy"
println "Uncle man, Uncle man, I dig ${lang}."
```

In Listing 3, I created a variable named `lang` and set its value to "Groovy." I printed out a String of type GString and asked it to substitute the spot after the word "dig" with the value of `${lang}`. If this were real life, the code would then print out "Uncle man, Uncle man, I dig Groovy." Pretty nifty, eh?

Run-time substitution is actually a common feature among dynamic languages, and, as usual, Groovy takes things one step further. Groovy's GStrings allow you to *autocall* methods on substituted values, which opens up a wide variety of possibilities when it comes to building dynamic text. For example, in Listing 4, I can call a method on my desired variable (in this case, the method `length()` on an

object of type `String`.

Listing 4. GString autocalling

```
def lang = "Groovy"
println "I dig any language with ${lang.size()} characters in its name!"
```

The code in Listing 4 will print out "I dig any language with 6 characters in its name!" In the next sections, I'll show you how to use Groovy's autocalling feature to enable some sophisticated features in your templates.

Groovy templates

Working with templates can be broken down to two primary tasks: First, you create a template; second, you provide the mapping code. Creating a template with Groovy's template framework is quite similar to creating a JSP, since you can reuse the syntax found in JSPs. The key to creating these templates is in defining the variables that will be substituted at run time. For example, in Listing 5, I've defined a template for creating `GroovyTestCases`.

Listing 5. A Template for creating GroovyTestCases

```
import groovy.util.GroovyTestCase
class <%=test_suite %> extends GroovyTestCase {
    <% test_cases.each{ tc ->
        println "\tvoid ${tc}() { } "
    }%>
}
```

The template in Listing 5 looks just like a JSP file because I've used the `<%` and `<%=` syntax. True to Groovy's flexibility, however, you're not restricted to a JSP syntax. You're also free to use Groovy's excellent `GStrings`, as shown in Listing 6.

Listing 6. GStrings in action

```
<person>
  <name first="${p.fname}" last="${p.lname}"/>
</person>
```

In Listing 6, I've created a simple template that represents an XML document defining a collection of `person` elements. You can see that the template expects some object, named `p`, that will have `fname` and `lname` properties.

Defining templates in Groovy is rather easy, and that's exactly the way it should be. Groovy templates aren't rocket science. They're simply a means for facilitating the

separation of a view from its model. The next step is to write the run-time mapping code.

Run-time mapping

Now that I've defined a template, I'd like to use it by mapping my defined variables to run-time values. As usual with Groovy, what might seem complicated is actually quite simple. All I need is a map whose keys are the variable names in the template and whose key values are the intended run-time values.

For example, if a simple template had a variable named `favlang`, I'd have to define a map with a key value of `favlang`. The key's value would be whatever I chose as my favorite scripting language (in this case, Groovy, of course).

In Listing 7, I've defined this simple template, and in Listing 8, I'll show you the corresponding mapping code.

Listing 7. Simple template to demonstrate mapping

```
My favorite dynamic language is ${favlang}
```

Listing 8 shows a simple class that does five things, two of which are important. Can you tell what they are?

Listing 8. Mapping values for a simple template

```
import groovy.text.Template
import groovy.text.SimpleTemplateEngine
class SimpleTemplate{
    static void main(String[] args) {
        def fle = new File("simple-txt.tmpl")
        def binding = [favlang: "Groovy"]
        def engine = new SimpleTemplateEngine()
        def template = engine.createTemplate(fle).make(binding)
        println template.toString()
    }
}
```

Mapping the values for the simple template in Listing 8 was surprisingly easy.

First, I created a `File` instance pointing to the template, `simple-txt.tmpl`.

Then I created a `binding` object, which is, in essence, a map. I mapped the value found in the template `favlang` to the `String` `Groovy`. This mapping is the first important step for using templates in Groovy or, for that matter, any language that has a template engine.

Next, I created an instance of `SimpleTemplateEngine`, which happens to be a particular implementation of the template engine framework in Groovy. I then passed the engine instance the template (`simple-txt.tpl`) and the binding object. Bringing together the template and its binding object is the second important step in Listing 8 and the crux of working with template engines. Internally, the framework will map the values found in the binding object with the names in the corresponding template.

My last step in Listing 8 was to print the output of the process. As you can see, creating a binding object and providing the proper mapping is a breeze, at least in this simple example. In the next section, I'll put Groovy's template engine to the test with a more complicated example.

A more complex template

In Listing 9, I've created a `Person` class to represent the person element defined in [Listing 6](#).

Listing 9. A Person class in Groovy

```
class Person{
    int age
    String fname
    String lname
}
```

In Listing 10, you can see the mapping code that maps an instance of the above-defined `Person` class.

Listing 10. Mapping a Person class with a template

```
import groovy.text.Template
import groovy.text.SimpleTemplateEngine
class TemplatePerson{
    static void main(String[] args) {
        def pers1 = new Person(age:12, fname:"Sam", lname:"Covery")
        def fle = new File("person_report.tpl")
        def binding = [p:pers1]
        def engine = new SimpleTemplateEngine()
        def template = engine.createTemplate(fle).make(binding)
        println template.toString()
    }
}
```

The above code looks familiar, doesn't it? In fact, it looks just like [Listing 8](#), but with an added line creating the `pers1` instance. Now, take a quick look at the template in [Listing 6](#) again. Do you see how the template references the properties `fname` and `lname`? What I did was create an instance of `Person` with the property `fname` set

to "Sam" and the property `lname` set to "Covery."

When the code in Listing 10 is run, the output will be XML defining the person element, as shown in Listing 11.

Listing 11. Person template output

```
<person>
  <name first="Sam" last="Covery"/>
</person>
```

Mapping a list

In [Listing 5](#) I defined a template for a `GroovyTestCase`. If you look at that template now, you'll notice that the definition has some logic for iterating over a collection. In Listing 12, you should see some similar code, yet this code has logic for mapping a list of test cases.

Listing 12. Mapping a list of test cases

```
import groovy.text.Template
import groovy.text.SimpleTemplateEngine

def fle = new File("unit_test.tpl")
def coll = ["testBinding", "testToString", "testAdd"]
def binding = [test_suite:"TemplateTest", test_cases:coll]
def engine = new SimpleTemplateEngine()
def template = engine.createTemplate(fle).make(binding)
println template.toString()
```

Looking at [Listing 5](#) shows that the template expects a list by the name of "test_cases" -- which I've defined in Listing 12 as `coll` containing three elements. I simply set `coll` to the "test_cases" key in the binding object and the code was good to go.

It should be clear by now that Groovy templates are easy to use. They also facilitate the ubiquitous MVC pattern and, even more importantly, they support refactoring into MVC code by embodying the view. In the next section, I'll show you how to refactor an example from a previous article, applying the lessons learned here.

Refactoring with templates

In the column on Ant scripting with Groovy, I wrote a simple utility that generated a checksum report of class files. If you remember, I clumsily coded the XML using `println`s. That code was so ugly even I had to admit it, as you can see for yourself by looking at Listing 13.

Listing 13. Smelly code

```
nfile.withPrintWriter{ pwriter ->
    pwriter.println("<md5report>")
    scanner.each{ f ->
        f.eachLine{ line ->
            pwriter.println("<md5 class='" + f.path + "' value='" + line + "'/>")
        }
    }
    pwriter.println("</md5report>")
}
```

To refresh your memory on the context, the code in Listing 13 takes some data and writes to a file (the `nfile` instance) with a `PrintWriter`. Notice how I hardcoded the view component of the report (the XML) inside a `println`. The problem with this approach is its inflexibility. If, at some point later, I needed to make a change, I'd have to go into the Groovy script's logic to do it. In an even worse scenario, imagine if a nonprogrammer wanted to make the change. The Groovy code would be so intimidating.

Moving the view portion of the script into a template would facilitate maintenance since changing the template would be a more natural process to anyone involved, so that's what I'll do here.

Defining the template

I'll start by defining the template -- it'll look much like the intended output with some logic to iterate over a collection of classes.

Listing 14. Refactoring old code into a template

```
<md5report>
<% clazzes.each{ clzz->
    println ""<md5 class="${clzz.name}" value="${clzz.value}"/>""
}%>
</md5report>
```

The template defined in Listing 14 looks similar to the template for `GroovyTestCases`, in that it includes logic for iterating over a collection. Note also that I mixed JSP syntax with `GStrings`.

Writing the mapping code

With the template defined, the next step is to write the run-time mapping code. I need to replace the old file-writing logic with code that builds a collection of `ChecksumClass` objects and then places those objects into a binding object.

The model then becomes the `ChecksumClass` defined in Listing 15.

Listing 15. CheckSumClass defined in Groovy

```
class CheckSumClass{
    String name
    String value
}
```

Class definitions are fairly easy in Groovy, no?

Creating a collection

Next, I need to refactor the section of code that previously wrote to a file -- this time with logic to populate a list with the new `CheckSumClass`, as shown in Listing 16.

Listing 16. Refactored code creating a collection of ChecksumClasses

```
def classez = []
scanner.each{ f ->
    f.eachLine{ line ->
        def iname = formatClassName(bsedir, f.path)
        classez << new CheckSumClass(name:iname, value:line)
    }
}
```

Listing 16 shows how easy it is to use Ruby-like syntax to add objects to lists -- and it's actually *quite* groovy. I first create lists with the `[]` syntax. I then use short-hand for loop notion followed by an iterator with a closure. The closure takes each line, which, in this case, is a checksum value, and creates an instance of the newly defined `CheckSumClass` (using Groovy's autogenerated constructor) and adds them to a collection. Not bad -- and it's also fun to write.

Adding the template mapping

The last thing I need to do is add the template engine-specific code. This code will perform the run-time mapping and write the corresponding formatted template to the original file, as shown in Listing 17.

Listing 17. Refactoring with template mapping

```
def fle = new File("report.tmpl")
def binding = ["clazzes": classez]
def engine = new SimpleTemplateEngine()
def template = engine.createTemplate(fle).make(binding)
nfile.withPrintWriter{ pwriter ->
    pwriter.println template.toString()
}
```

By now, the code in Listing 17 should be old hat to you. I take the list from Listing 16 and place it in the binding object. I then take the `nfile` object and write the

corresponding output from the mapped template from [Listing 14](#).

Before I put this all together in Listing 18, you might want to take one last look at the smelly code I started out with, back in [Listing 13](#). And here's the newly refactored code for contrast:

Listing 18. Look, Ma! Less smelly code!

```
void buildReport(String[] dirs, String todir){
    def ant = new AntBuilder()
    dirs.each{bsedir ->
        def scanner = ant.fileScanner {
            fileset(dir:bsedir) {
                include(name:"**/*.class.md5.txt")
            }
        }

        def rdir = todir + File.separator + bsedir + File.separator + "xml" + File.separator
        def file = new File(rdir)

        if(!file.exists()){
            ant.mkdir(dir:rdir)
        }

        def nfile = new File(rdir + File.separator + "checksum.xml")

        //newly refactored code using templates
        def classez = []
        scanner.each{ f ->
            f.eachLine{ line ->
                def iname = formatClassName(bsedir, f.path)
                classez << new CheckSumClass(name:iname, value:line)
            }
        }
        def file = new File("report.tmpl")
        def binding = ["clazzes": classez]
        def engine = new SimpleTemplateEngine()
        def template = engine.createTemplate(file).make(binding)
        nfile.withPrintWriter{ pwriter ->
            pwriter.println template.toString()
        }
    }
}

String formatClassName(String dirName, String className){
    def paths = dirName.split("\\\\")
    return paths.join(".") + "." + className
}
```

Now, I've never claimed to write beautiful code, but this code certainly isn't as *ugly* as my older code was. And to think all I've done is to replace a few smelly old `println`s with Groovy's much nicer template code. (Adept refactoring folks will note that I could apply *Extract Method* as a further refinement to the code.)

Conclusion

In this month's lesson, I hope I've expanded your, um, *view* of Groovy. To put it less

punningly, Groovy's template framework is a slick alternative for plain-Jane Java coding when you need to quickly knock out simple applications that require a view aspect. Templates are a beautiful abstraction and can seriously facilitate long-term maintenance of your applications if used properly.

Next month, I'm going to show you how to use Groovy to build Web applications with Groovlets. Until then, happy template development with Groovy.

Downloads

Description	Name	Size	Download method
Sample code	j-pg02155.zip	5KB	HTTP

[Information about download methods](#)

About the authors

Andrew Glover

Andrew Glover is a developer, author, speaker, and entrepreneur. He is the founder of the [easyb](#) Behavior-Driven Development (BDD) framework and is the co-author of three books: [Continuous Integration](#), [Groovy in Action](#), and [Java Testing Patterns](#). He teaches a wide variety of Groovy-, Grails-, and testing-related classes at [ThirstyHead.com](#). You can keep up with Andy at [thediscoblog.com](#) where he routinely blogs about software development.

Scott Davis

Scott Davis is an internationally recognized author, speaker, and software developer. He is the founder of [ThirstyHead.com](#), a Groovy and Grails training company. His books include [Groovy Recipes: Greasing the Wheels of Java](#), [GIS for Web Developers: Adding Where to Your Application](#), [The Google Maps API](#), and [JBoss At Work](#). He writes two ongoing article series for IBM developerWorks: [Mastering Grails](#) and [Practically Groovy](#).